

ECE 250 *Algorithms and Data Structures*

Hash functions

Douglas Wilhelm Harder, M.Math. LEL
Department of Electrical and Computer Engineering
University of Waterloo
Waterloo, Ontario, Canada

ece.uwaterloo.ca
dwharder@alumni.uwaterloo.ca

© 2006-2013 by Douglas Wilhelm Harder. Some rights reserved.

WATERLOO
ENGINEERING

Definitions

What is a hash of an object?

From Merriam-Webster:

a restatement of something that is already known

The ultimate goal is to map onto an integer range

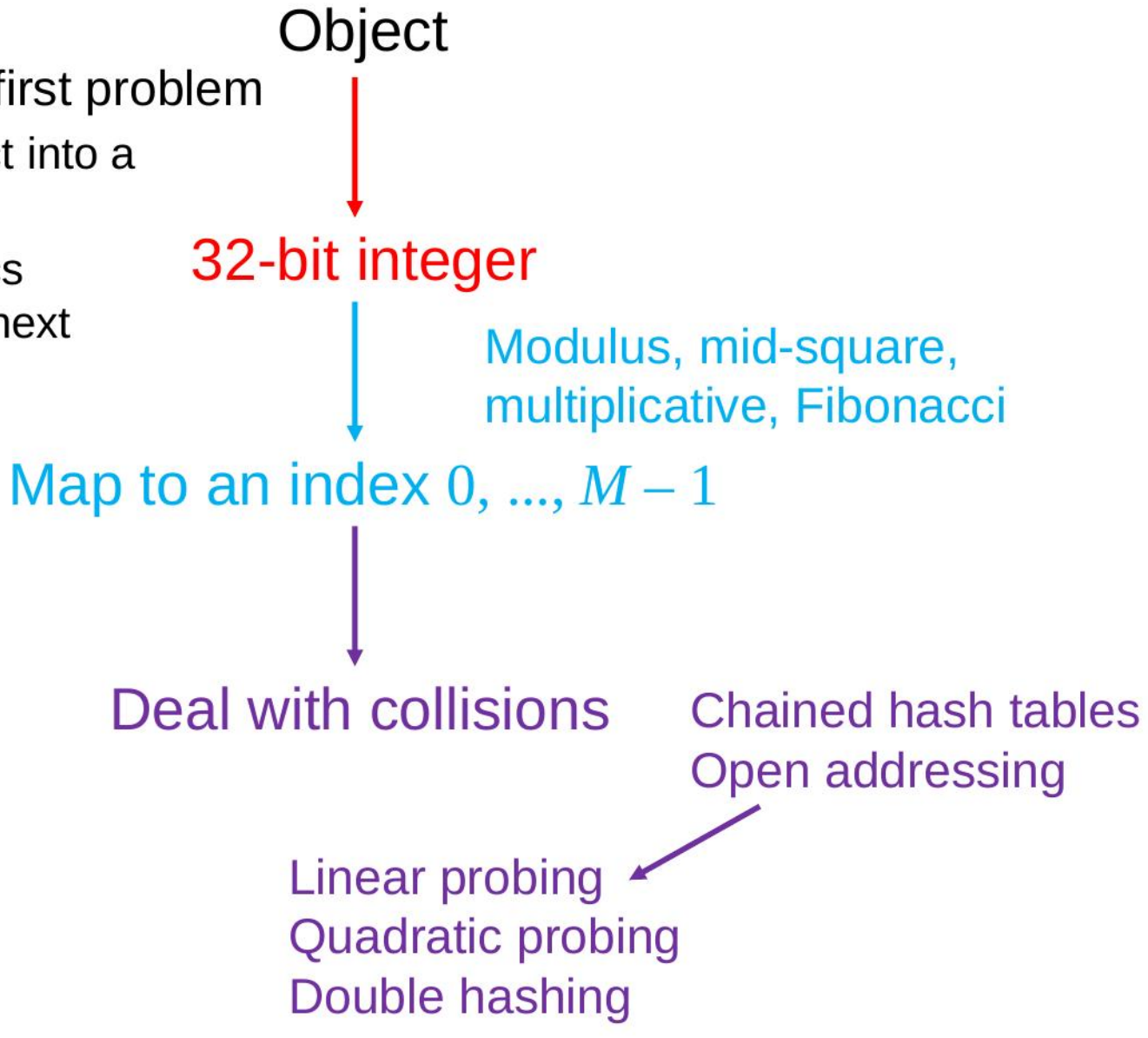
$$\mathbf{0, 1, 2, \ldots, M - 1}$$

9.2.1

The hash process

We will look at the first problem

- Hashing an object into a 32-bit integer
- Subsequent topics will examine the next steps



9.2.2

Properties

Necessary properties of such a hash function h are:

1a. Should be fast: ideally $\Theta(1)$

1b. The hash value must be *deterministic*

- It must always return the same 32-bit integer each time

1c. Equal objects hash to equal values

- $x = y \implies h(x) = h(y)$

1d. If two objects are randomly chosen, there should be only a one-in-

2^{32} chance that they have the same hash value

9.2.4

Predetermined hash functions

For example, an auto-incremented static member variable

```
class Class_name {
   private:
      unsigned int hash_value;
      static unsigned int hash_count;
   public:
      Class_name();
      unsigned int hash() const;
};

unsigned int Class_name::hash_count = 0;
```

```
Class_name::Class_name() {
   hash_value = hash_count;
   ++hash_count;
}

unsigned int Class_name::hash() const {
   return hash_value;
}
```

9.2.4.1

Predetermined hash functions

Predetermined hash values give each object a unique hash value

This is not always appropriate:

- Objects which are conceptually equal:

```
Rational x(1, 2);
Rational y(3, 6);
```

- Strings with the same characters:

```
string str1 = "Hello world!";
string str2 = "Hello world!";
```

These hash values must depend on the member variables

- Usually this uses arithmetic functions

9.2.5.1

Rational number class

What if we just add the numerator and denominator?

```
class Rational {
   private:
      int numer, denom;
   public:
      Rational( int, int );
};

unsigned int Rational::hash() const {
   return static_cast<unsigned int>( numer ) +
      static_cast<unsigned int>( denom );
}
```

9.2.5.1

Rational number class

We could improve on this: multiply the denominator by a large prime:

```
class Rational {
   private:
      int numer, denom;
   public:
      Rational( int, int );
};

unsigned int Rational::hash() const {
   return static_cast<unsigned int>( numer ) +
      429 496 751*static_cast<unsigned int>( denom );
}
```

9.2.5.1

Rational number class

For example, the output of

```
int main() {
    cout << Rational(  0,   1 ).hash() << endl;
    cout << Rational(  1,   2 ).hash() << endl;
    cout << Rational(  2,   3 ).hash() << endl;
    cout << Rational( 99, 100 ).hash() << endl;

    return 0;
}
```

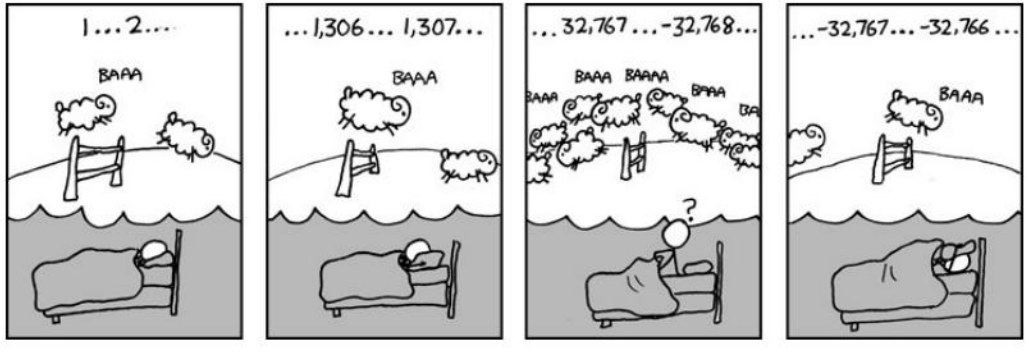
is

429 496 751

858 993 503

1 288 490 255

2239



http://xkcd.com/571/

Recall that arithmetic operations wrap on overflow

9.2.5.1

Rational number class

This hash function does not generate unique values

- The following pairs have the same hash values:

0/1	1327433019/800977868
1/2	534326814/1480277007
2/3	820039962/1486995867

- Finding rational numbers with matching hash values is very difficult:
- Finding these required the generation of $1\,500\,000\,000$ random rational numbers
- It is fast: $\Theta(1)$
- It does produce an even distribution

9.2.5.1

Rational number class

Problem:

- The rational numbers 1/2 and 2/4 have different values
- The output of

```
int main() {
    cout << Rational( 1, 2 ).hash();
    cout << Rational( 2, 4 ).hash();
    return 0;
}
```

is

```
858 993 503
1 717 987 006
```

9.2.5.1

Rational number class

Solution: divide through by the greatest common divisor

```
Rational::Rational( int a, int b ):numer(a), denom(b) {
   int divisor = gcd( numer, denom );
   numer /= divisor;
   denom /= divisor;
}
```

```
int gcd( int a, int b) {
   while( true ) {
      if ( a == 0 ) {
         return (b >= 0) ? b : -b;
      }

      b %= a;

      if ( b == 0 ) {
         return (a >= 0) ? a : -a;
      }
      a %= b;
   }
}
```

9.2.5.1

Rational number class

Problem:

- The rational numbers $\frac{1}{2}$ and $\frac{-1}{-2}$ have different values
- The output of

```
int main() {
    cout << Rational(  1,  2 ).hash();
    cout << Rational( −1, −2 ).hash();
    return 0;
}
```

is

```
858 993 503
3 435 973 793
```

9.2.5.1

Rational number class

Solution: define a normal form

- Require that the denominator is positive

```
Rational::Rational( int a, int b ):numer(a), denom(b) {
    int divisor = gcd( numer, denom );
    divisor = (denom >= 0) ? divisor : -divisor;
    numer /= divisor;
    denom /= divisor;
}
```

9.2.5.3

String class

Two strings are equal if all the characters are equal and in the identical order

A string is simply an array of bytes:

- Each byte stores a value from 0 to 255

Any hash function must be a function of these bytes

9.2.5.3.1

String class

We could, for example, just add the characters:

```
unsigned int hash( const string &str ) {
   unsigned int hash_vaalue = 0;

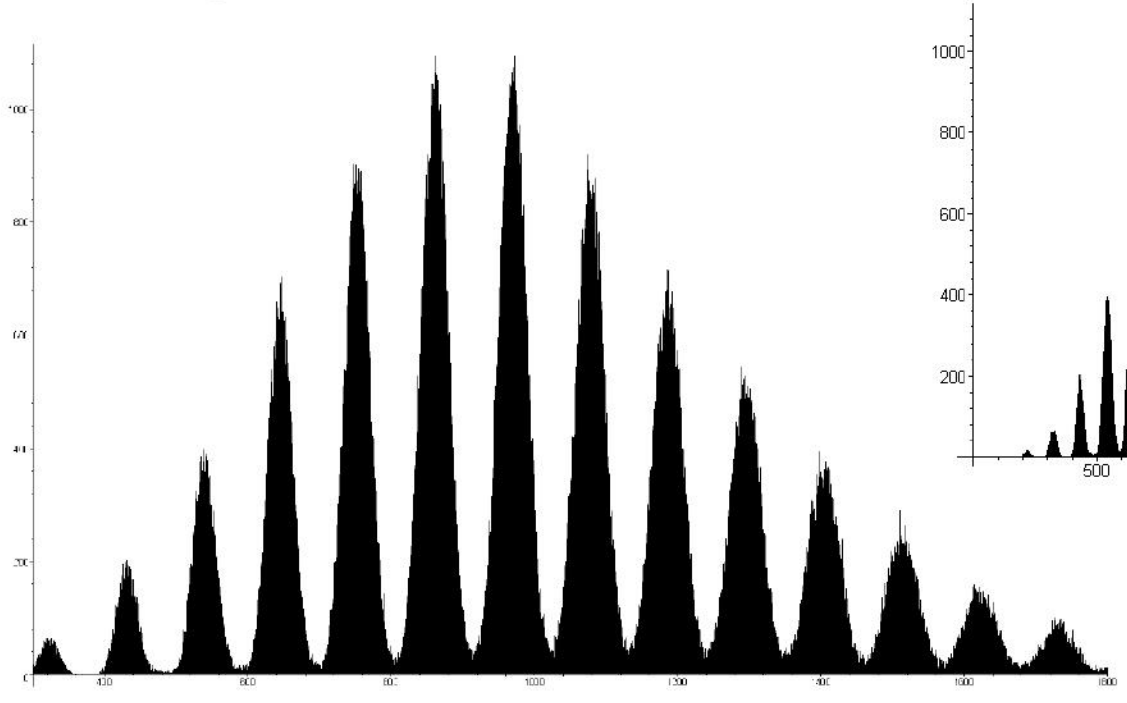
   for ( int k = 0; k < str.length(); ++k ) {
      hash_value += str[k];
   }

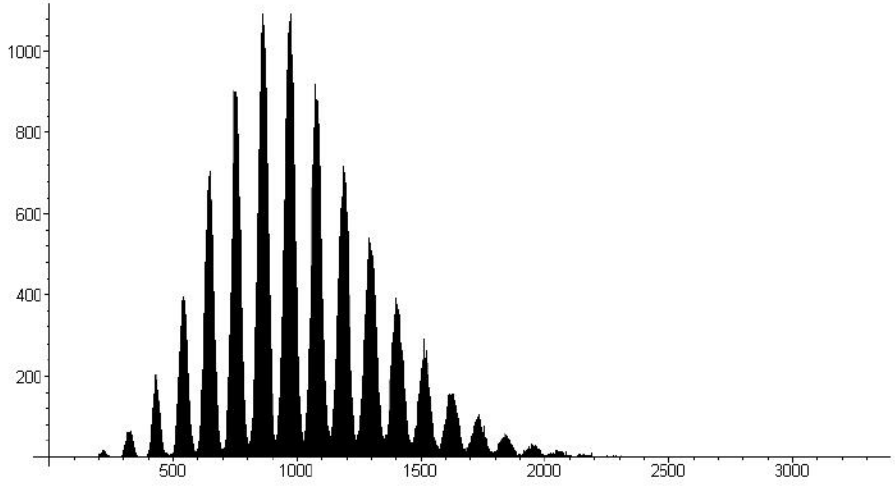
   return hash_value;
}
```

String class

Not very good:

- Slow run time: $\Theta(n)$
- Words with the same characters hash to the same code:
  - "form" and "from"
- A poor distribution, *e.g.*, all words in Moby™ Words II by Grady Ward (single.txt) Project Gutenberg):





String class

Let the individual characters represent the coefficients of a polynomial in x:

$$p(x) = c_0 x^{n-1} + c_1 x^{n-2} + \cdots + c_{n-3} x^2 + c_{n-2} x + c_{n-1}$$

Use Horner's rule to evaluate this polynomial at a prime number, *e.g.*, $x = 12347$:

```
unsigned int hash( string const &str ) {
   unsigned int hash_value = 0;

   for ( int k = 0; k < str.length(); ++k ) {
      hash_value = 12347*hash_value + str[k];
   }

   return hash_value;
}
```

String class

Use characters in locations $2^k - 1$ for $k = 0, 1, 2, ...$:

"**A**_E**l**ber**e**th Gilt**h**oniel,\n
Silivren_penna miriel\n
O menal aglar elen**a**th!\n
Na-chaered palan-diriel\n
O galadhremmin ennorath,\n
Fanuilos, **l**e linnathon\n
nef aear, si nef aearon!"

J.R.R. Tolkien

String class

The run time is now $\Theta(\ln(n))$:

```
unsigned int hash( const string &str ) {
   unsigned int hash_value = 0;

   for ( int k = 1; k <= str.length(); k *= 2 ) {
      hash_value = 12347*hash_value + str[k - 1];
   }

   return hash_value;
}
```

Note: this cannot be used if you require a cryptographic hash function or *message digest*